

بسم الله الرحمن الرحيم

گردآورنده: محمدصادق قیاس‌سوند

کد دانشجویی: ۴۰۲۱۲۳۱۰۶۹

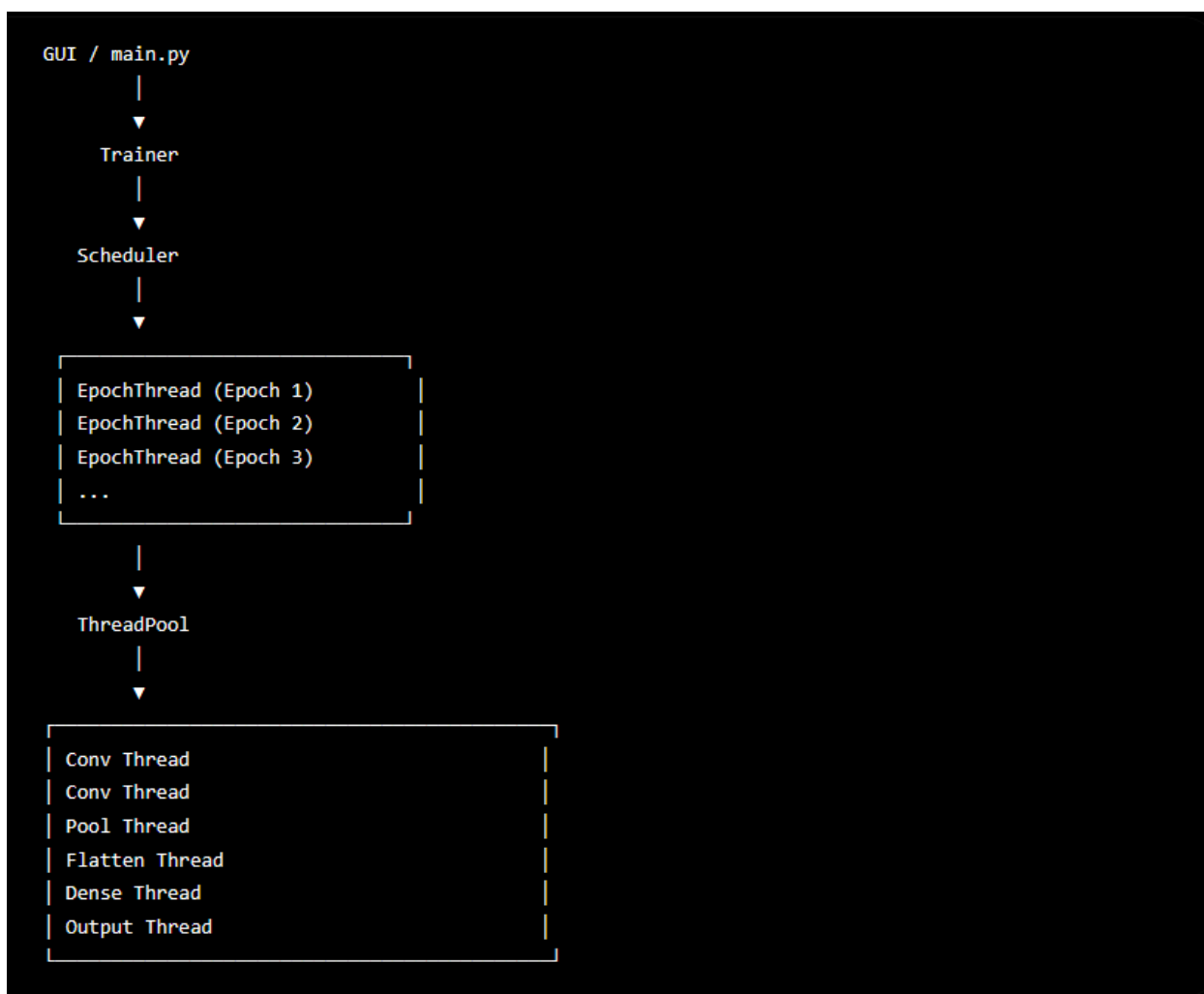
استاد: دکتر کوثر طالبی

۱. معماری کامل نخها (Thread Architecture)

معماری پروژه به صورت چندلایه طراحی شده است تا مفاهیم سیستم عامل از منطق شبکه عصبی جدا باشند. به همین دلیل کل بخش مدیریت نخها در پوشه `os_layer` قرار گرفته و بخش آموزش شبکه عصبی تنها از این زیرسیستم استفاده می کند.

به طور کلی ساختار اجرای برنامه به صورت زیر است:

(در این معماری، هر بخش مسئولیت مشخصی دارد و از اصل **Single Responsibility** پیروی می کند.)



۲. محتویات پوشه `os_layer`:

(تمامی مفاهیم سیستم عامل داخل این پوشه پیاده سازی شده اند.)

۲-۱: ThreadPool

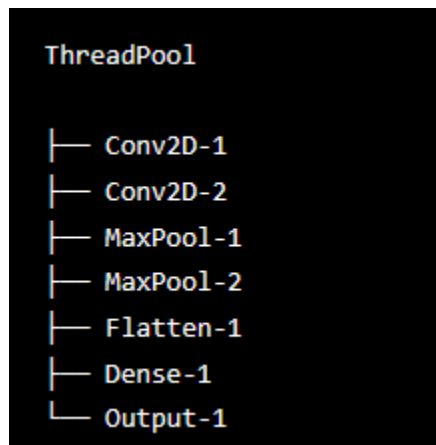
این کلاس مسئول ایجاد و مدیریت نخ های مربوط به لایه های CNN است.

در زمان شروع برنامه، برای هر لایه شبکه عصبی یک یا چند Worker ساخته می شود.

مثلاً اگر شبکه شامل لایه های زیر باشد:

Conv2D – MaxPool – MaxPool – Flatten – Dense – Output – Conv2D

ThreadPool نیز برای هر کدام نخ ایجاد می کند و ساختار آن تقریباً به صورت روبرو است.



وظایف انبار نخ عبارت اند از:

- ایجاد نخ ها
- شروع اجرای آنها
- ارسال Task به نخ مناسب
- انتظار برای پایان تمام Task ها
- خاموش کردن تمام نخ ها

به این ترتیب Trainer هیچ اطلاعی از نحوه مدیریت نخ ها ندارد.

۲-۲: LayerThread

هر نمونه از این کلاس نماینده یک نخ واقعی سیستم عامل است. هر نخ دارای نسبت های زیر می باشد:

- صف اختصاصی
- سمافور
- حلقه اجرای دائمی
- Shutdown Event

هر نخ دائماً منتظر دریافت Task جدید می ماند. و ساختار آن به شکل زیر است:

while True

wait(semaphore)

Task دریافت

اجرا

Task پایان

بنابراین نخ ها فقط یک بار ساخته می شوند و برای هر دوره (Epoch) مجدداً ایجاد نمی شوند، این موضوع هزینه ایجاد نخ را کاهش می دهد.

۲-۳- EpochThread:

هر دوره آموزش توسط یک نخ مستقل اجرا می شود، اگر تعداد دوره ها برابر ۵ باشد، Scheduler پنج نمونه از این کلاس می سازد. هر EpochThread مراحل زیر را انجام می دهد:

۱. ارسال Task به تمام LayerThread ها ->
۲. انتظار برای پایان اجرای لایه ها ->
۳. اجرای یک دوره واقعی TensorFlow ->
۴. ذخیره Accuracy و Loss ->
۵. ثبت زمان اجرا

۲-۴- Scheduler:

Scheduler مسئول مدیریت نخ های دوره است که وظایف آن به شرح زیر است:

- ثبت نخ ها
- شروع کردن همه آنها (start)
- صبر کردن تا پایان (wait)
- چاپ Summary

در حقیقت Scheduler نقش زمانبند سیستم عامل را شبیه سازی می کند.

۲-۵-SynchronizationManager:

این کلاس مسئول هماهنگی بین دوره ها است. برای جلوگیری از اجرای همزمان بخش حساس آموزش، از یک Lock استفاده شده است. به این ترتیب هیچ دو دوره ای به طور همزمان وارد بخش حساس TensorFlow نمی شوند.

۲-۶-ThreadStatus:

در این کلاس برای هر نخ وضعیت آن نگهداری می شود.

>در صورت بروز خطا- CREATED -> WAITING -> RUNNING -> FINISHED

FAILED

۳- پاسخ صریح به سوالات آخر pdf تشریح پروژه

۱. نخهای اجرایی مختلفی برای هر کدام از لایه های شبکه عصبی و هر کدام از دوره های اجرایی آن ایجاد کنید.

۲. حداقل تعداد سمافور الزم را به کار ببرید تا داده های x_train و y_train که توسط هر نخ مجزا استفاده میشوند و همچنین محاسبات نهایی، دچار مشکل نشوند.

۱-۳- سوال اول:

(این بخش در دو فایل پیاده سازی شده است.)

بخش اول در threadpool:

در این فایل برای هر لایه از شبکه عصبی یک LayerThread ساخته می شود. به عنوان نمونه:

Conv2D Thread - Pooling Thread - Flatten Thread - Dense Thread - Output Thread

بخش دوم در epoch_thread.py:

در این فایل برای هر دوره یک نخ مستقل ایجاد می‌شود. Trainer هنگام مقداردهی اولیه به تعداد دوره‌های تعیین‌شده، نمونه‌های EpochThread را می‌سازد و در Scheduler ثبت می‌کند. بنابراین پروژه دارای دو سطح نخ است:

- نخ‌های مربوط به لایه‌ها
- نخ‌های مربوط به دوره‌ها
-

۳-۲- سوال دوم:

(این قسمت هم در دو فایل پیاده‌سازی شده است.)

Semaphore اول در layer_thread.py:

هر LayerThread دارای یک Semaphore اختصاصی است.

```
self.semaphore = Semaphore(۰)
```

که وظیفه آن این است که هر نخ تا زمانی که Task جدید دریافت نکند، در حالت انتظار باقی می‌ماند. وقتی ThreadPool یک Task ارسال می‌کند:

```
submit() -> Queue.put() -> Semaphore.release()
```

نخ بیدار شده و ماموریت را اجرا می‌کند.

Semaphore دوم (هماهنگی بخش حساس):

در کلاس SynchronizationManager از یک قفل استفاده شده است. این قفل اجازه نمی‌دهد دو دوره همزمان وارد بخش آموزش TensorFlow شوند. در نتیجه:

- داده‌های x_train
- داده‌های y_train

- وزن‌های مدل

- Gradient ها

به صورت همزمان توسط دو نخ تغییر نمی‌کنند.

این **Semaphore** لازم بود چرا که اگر این هماهنگی انجام نمی‌شد، دو دوره ممکن بود همزمان عملیات زیر را انجام دهند:

Forward Pass -> Gradient Update -> Weight Update

در نتیجه وزن‌های شبکه توسط چند نخ به طور همزمان تغییر می‌کرد و خروجی آموزش غیرقابل پیش‌بینی می‌شد. (Race Condition) به همین دلیل بخش بحرانی (Critical Section) با مکانیزم همگام‌سازی محافظت شده است.

جمع‌بندی معماری

به طور خلاصه، معماری بخش سیستم‌عامل پروژه به صورت زیر است:

Trainer -> Scheduler -> Epoch Threads -> ThreadPool -> Layer Threads ->

(Semaphore + Queue) -> TensorFlow Training

در طراحی ارائه شده هر بخش وظیفه مشخصی دارد:

- **Trainer** کل فرایند آموزش را مدیریت می‌کند .
- **Scheduler** چرخه عمرنخ‌های دوره را کنترل می‌کند .
- **EpochThread** اجرای یک دوره آموزش را بر عهده دارد .
- **ThreadPool** نخ‌های مربوط به لایه‌های شبکه را ایجاد و مدیریت می‌کند .
- **LayerThread** وظیفه‌های مربوط به هر لایه را از صف دریافت و اجرا می‌کند .
- **SynchronizationManager** با استفاده از همگام‌سازی، از بروز شرایط رقابتی - (Race Condition) در بخش‌های حساس جلوگیری می‌کند.

و در آخر شایان ذکر میدانم که این مهم هم خدمتتان عرض کنم، در ساختار و مجموعه فایل های گردآمده ارائه شده، هدف انجام پروژه به صورت واقعی در عین حالی که خواسته های استاد مربوطه را اغنا می کند ...

با امید ایرانی سرافراز بر قله های علم و دانش

محمدصادق قیاسوند